Digital Signal Processing
Systems
Script: Chapter 2 (Systems)

Exercise Sheet 2
with solutions

Learning goals

- Use the operator view $y = S\{x\}$ and decide for a given system whether it is static or dynamic (memory), causal, linear and time-invariant.
- Understand how a two-point lambda controller with integrator works, why it produces a limit cycle, and predict its period from the loop dead time.
- *Bonus (optional):* design numerical test harnesses that *reveal* a property and know their pitfalls (impulse-only tests, small amplitudes, boundary effects); model engine subsystems as dynamic systems: inertia (first-order lag), wall film, transport delay, switching sensor.

Definitions used on this sheet

For a system $y = S\{x\}$ (continuous or discrete time):

- *memoryless (static)*: $y(t_0)$ depends only on $x(t_0)$;
- *causal*: $y(t_0)$ depends only on $x(t)$, $t \le t_0$; equivalently, if $x_1(t) = x_2(t)$ for $t < t_0$, then $y_1(t) = y_2(t)$ for $t < t_0$;
- *linear*: $S\{ax_1 + bx_2\} = aS\{x_1\} + bS\{x_2\}$ for all inputs and constants (superposition = additivity + homogeneity);
- *time-invariant*: $y(t) = S\{x(t)\} \Rightarrow y(t - t_0) = S\{x(t - t_0)\}$ for every shift t_0 (discrete time: integer shifts n_0).

Linearity and time invariance are previewed here; Chapter 3 shows why their combination (LTI) is so powerful.

Exercise 2.1 Properties by hand

PAPER

a) Decide for each system whether it is memoryless, causal, linear and time-invariant (table with yes/no):

S1: $y(t) = 2x(t)$	S2: $y(t) = x^2(t)$	S3: $y[n] = x[n+1] - x[n]$
S4: $y[n] = \frac{1}{3}(x[n] + x[n-1] + x[n-2])$	S5: $y(t) = x(t - T_d)$, $T_d > 0$	S6: $y(t) = \lvert x(t)\rvert$
S7: $y(t) = \int_{-\infty}^{t} x(\tau)\,\mathrm{d}\tau$	S8: $y[n] = n\,x[n]$	S9: $y[n] = x[n] + 0.5$
S10: $y[n] = x[2n]$	S11: $n_k = 1/T_k$ (Sheet 1)	S12: $U = 0.5 + 0.4\tanh\frac{1-\lambda}{0.005}$

b) Prove your answers formally for S6 (linearity), S8 (time invariance) and S10 (causality and time invariance). For S9, show that the system is not linear, but that the *difference* of two outputs depends linearly on the difference of the inputs (*incrementally linear*).

c) An engineer computes the mean speed of one crank revolution as the arithmetic mean of the 58 per-tooth speeds $1/T_k$, treating the gap period like a normal tooth period. Compute the relative error at constant speed. After correcting the gap ($3/T_{\text{gap}}$), is the arithmetic mean of the per-tooth speeds equal to $60/T_{\text{rev}}$? Which property of S11 is responsible, and what is the error for a speed fluctuation of $\pm 1\,\%$ (sinusoidal)?

Solution

a) (M = memoryless, C = causal, L = linear, TI = time-invariant)

system	M	C	L	TI	remark
S1 $2x$	yes	yes	yes	yes	amplifier
S2 x^2	yes	yes	no	yes	$S\{2x\} = 4S\{x\}$
S3 $x[n+1] - x[n]$	no	no	yes	yes	forward difference uses the future
S4 3-point MA	no	yes	yes	yes	memory of 2 samples
S5 delay	no	yes	yes	yes	transport delay (lambda!)
S6 $\lvert x\rvert$	yes	yes	no	yes	rectifier
S7 integrator	no	yes	yes*	yes	*initially at rest
S8 $n\,x[n]$	yes	yes	yes	no	gain changes with time
S9 $x + 0.5$	yes	yes	no	yes	affine, $S\{0\} \neq 0$
S10 $x[2n]$	no	no	yes	no	down-sampler
S11 $1/T$	yes	yes	no	yes	speed from tooth period
S12 tanh	yes	yes	no	yes	switching lambda sensor

b) *S6*: choose $x_1 = 1$, $a = -1$: $S\{-x_1\} = 1 \neq -S\{x_1\} = -1$, so homogeneity fails (additivity fails too: $x_1 = 1, x_2 = -1$ gives $0 \neq 2$). *S8*: input $x_2[n] = x[n - n_0]$ gives $y_2[n] = n\,x[n - n_0]$, whereas the shifted output is $y[n - n_0] = (n - n_0)\,x[n - n_0]$; they differ for $n_0 \neq 0 \Rightarrow$ time-variant (but linear: $n(ax_1 + bx_2) = a\,nx_1 + b\,nx_2$). *S10*: $y[1] = x[2]$ depends on a future input $\Rightarrow$ non-causal (for $n > 0$), and the output at n depends on $x[2n] \neq x[n] \Rightarrow$ memory. Shift: $x_2[n] = x[n-1]$ gives $y_2[n] = x[2n-1]$, but $y[n-1] = x[2n-2] \Rightarrow$ time-variant. (Only shifts by even n_0 commute with S10 – in the output domain by $n_0/2$; the down-sampler is "periodically time-variant".) *S9*: $S\{ax\} = ax + 0.5 \neq a(x + 0.5)$, and $S\{0\} = 0.5 \neq 0$ (a linear system must map 0 to 0). But $y_1 - y_2 = x_1 - x_2$: the deviation from the operating point is transferred linearly. This is how all "linearised" engine models around an operating point work.

c) Treating the gap like a tooth, its contribution is $1/T_{\text{gap}} = n/3$:

$$\bar{n} = \frac{57\,n + n/3}{58} = 0.98851\,n, \qquad \text{error } -1.15\,\%.$$

With gap correction all 58 values equal n at constant speed, but under a speed fluctuation the arithmetic mean of $1/T_k$ is *not* $1/\overline{T}$: S11 is nonlinear (convex), and by Jensen's inequality $\overline{1/T} \geq 1/\overline{T}$. With $n(\theta) = n_0(1 + \varepsilon \sin 2\theta)$ the per-tooth speeds average to n_0 (teeth are equally spaced in *angle*), whereas $60/T_{\text{rev}} = n_0/\overline{(1 + \varepsilon \sin)^{-1}} \approx n_0(1 - \varepsilon^2/2)$: the relative difference is $\varepsilon^2/2 = 5 \cdot 10^{-5}$ (0.15 rpm at 3000 rpm – exactly the offset seen on Sheet 1). The physically correct mean speed over a revolution is $60/T_{\text{rev}}$ (angle travelled / time); averaging in the period domain (sum of the T_k) and inverting once gives it exactly.

Exercise 2.2 Closed lambda loop

WOKWI

Project `code/wokwi/sheet02_lambda/`: Arduino Uno and the custom chip `lambda-probe`. Pin 9 sends the fuel command as PWM (`analogWrite`, 490 Hz, duty $= q/2$), A0 reads the sensor voltage `O2`, A1 the ground truth `LAM` $= \lambda/2$. The chip's sliders set the unmetered air (slider $- 10$ in %, default $+5$ %) and the transport delay (default 200 ms).

How a two-point controller works. The goal is the stoichiometric mixture $\lambda = 1$. The ECU does not know the fuel quantity q^* that achieves it, because part of the air is not measured (leaks, sensor and injector tolerances, ageing): with unmetered air a, $\lambda = (1 + a)/q$ and $q^* = 1 + a$. The switching sensor (S12 of Exercise 2.1) delivers only *one bit* of information: $U > 0.45$ V means "rich" ($\lambda < 1$, too much fuel), $U < 0.45$ V means "lean" ($\lambda > 1$, too little fuel) – but not *how* rich or lean. The controller therefore uses the simplest possible rule. In every controller period $T_s = 10$ ms:

- sensor rich $\Rightarrow$ decrease q by rT_s (ramp down with slope $-r$),

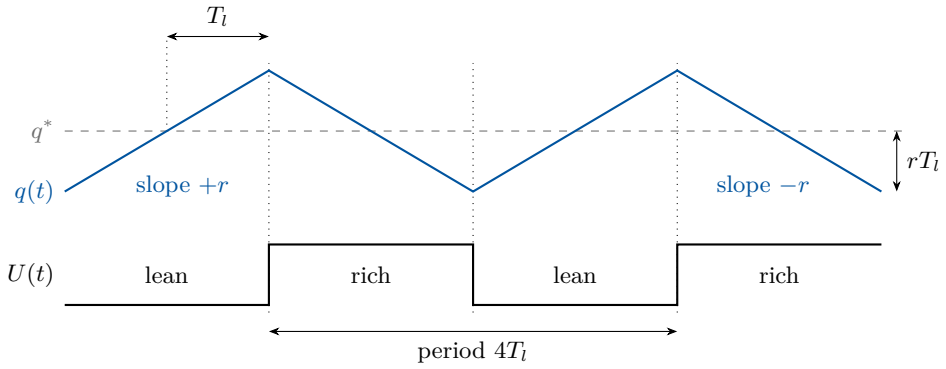
- sensor lean ⇒ increase q by rT_s (ramp up with slope $+r$),
- optionally, at the moment the sensor *switches*, an additional jump of q by P towards the other side ($-P$ on lean→rich, $+P$ on rich→lean).

"Two-point" means that the controller output (here the slope $\dot{q}$) takes only two values, $\pm r$, like a relay or a thermostat. "With integrator" means that q is the integral of this output: as long as the sensor reports the same state, q keeps moving in the same direction, so it always moves towards q^*, whatever the value of a. On average the controller therefore finds q^* by itself (integral action).

Why it oscillates. The sensor sees a change of q only after the transport delay T_d and the two lags (wall film $\tau_f = 20\,\text{ms}$, sensor $\tau_s = 30\,\text{ms}$), i.e. after an effective dead time $T_l \approx T_d + \tau_f + \tau_s$. When q crosses q^*, the sensor still reports the old state for T_l, and q keeps ramping in the wrong direction: it overshoots by rT_l. After the switching, q needs another T_l to return to q^*, and the sensor sees this crossing only T_l later. Each half period therefore lasts $2T_l$:

$$T_{\text{period}} \approx 4\,T_l, \qquad q = q^* \pm rT_l, \qquad \lambda \approx 1 \mp \frac{rT_l}{q^*}.$$

The oscillation neither grows nor decays – a *limit cycle*. The jump P removes part of the overshoot immediately after each switching, so q reaches q^* earlier (by P/r); the period shortens to $4T_l - 2P/r$.



Limit cycle without jump ($P = 0$): U is the sign of $q - q^*$, delayed by T_l.

a) Implement the two-point controller with integrator ($r = 0.2/\text{s}$) in `loop()`. Start the simulation and observe the limit cycle in the Serial Monitor (`u_o2`, `lambda`, `q`). Copy 5 s of data into a spreadsheet (or into JSLinux, see the bonus section) and plot λ and U; compare with Fig. 1.3 of the script and with the sketch above. Predict period and λ range for $T_d = 200\,\text{ms}$ and check them.

b) Measure the oscillation period (column `period_ms`) for transport delays of 100, 200, 400 and 800 ms and plot period over T_d. Determine slope and intercept and interpret them.

c) Move the air slider from +5 % to −10 % and to +10 %. Describe the transient and the new mean value of q.

d) Add the jump `PJUMP` = 0.03, then reduce `RATE` to 0.1 with `PJUMP` = 0.025. Compare period and λ amplitude. Why can no two-point controller make the oscillation disappear, and what would a continuous (PI) lambda controller need?

Solution

a) Controller (`code/solutions/wokwi/sheet02_lambda/sketch.ino`):

```
    /* ---- two-point controller with integrator ---------------------- */
    bool rich = u > U_THR;
    if (rich != rich_old) {
        q += rich ? -PJUMP : PJUMP;                    /* P jump */
```

```
        if (rich) {                                          /* lean -> rich */
            unsigned long now = millis();
            if (t_switch) period_ms = now - t_switch;
            t_switch = now;
        }
    }
    q += (rich ? -RATE : RATE) * (TS_US * 1e-6f);           /* I ramp */
    if (q < 0.5f) q = 0.5f;
    if (q > 1.5f) q = 1.5f;
    rich_old = rich;

    /* duty = q / 2, 8-bit PWM */
    analogWrite(INJ_PIN, (int)(127.5f * q + 0.5f));
```

The sensor voltage switches between $\approx 0.1\,\mathrm{V}$ and $0.9\,\mathrm{V}$ (ADC values ≈20 and 184), almost rectangular as in Fig. 1.3, while λ oscillates triangularly (ramps of q, smoothed by the wall film) between about 0.96 and 1.05; q is a triangle around 1.05 with steps of $2/255 = 0.0078$ (8-bit PWM). The ground-truth λ read via A1 has a resolution of only $2 \cdot 4.9\,\mathrm{mV} \approx 0.01$, so its extreme values scatter by ± 0.01. Expected period at $T_d = 200\,\mathrm{ms}$: $1.0\,\mathrm{s}$ ($4T_l = 4 \cdot (200 + 50)\,\mathrm{ms}$; the offline simulation of Bonus Exercise 2.B2 c) with the same model as the chip gives 0.98 s), and $\lambda \approx 1 \mp 0.2 \cdot 0.25/1.05 = 1 \mp 0.048$. The first period after the start is longer, because q starts at 1.0 and must first ramp to $q^* = 1.05$.

b) Expected values (the reference sketch run on the PC against a replica of the chip model): 0.58 s, 1.00 s, 1.80 s, 3.40 s for $T_d = 100$, 200, 400, 800 ms, with λ (from A1) in 0.978…1.026, 0.958…1.046, 0.919…1.095, 0.860…1.193. The offline model of Bonus Exercise 2.B2 gives up to 20 ms less, because there the controller sees the sensor value of the current millisecond (no extra sampling delay). Period over T_d is a straight line with slope 4.0 (each switching needs the signal to travel through the delay twice per half period) and intercept $\approx 0.18\,\mathrm{s} = 4 \cdot 45\,\mathrm{ms}$ (the two lags, 20 ms wall film + 30 ms sensor, minus the early switching of the sensor). In Wokwi the values may deviate by one controller period (10 ms) per switching.

c) At −10 % air the stoichiometric quantity drops to $q^* = 0.90$: the mixture is suddenly rich ($\lambda = 0.9/1.05$), the sensor stays at 0.9 V and the integrator ramps q down with 0.2 s, i.e. it needs $\approx (1.05 - 0.90)/0.2 = 0.75\,\mathrm{s}$ plus the dead time before the limit cycle resumes around $q = 0.90$. Simulation (`air_step()` in the reference solution of Bonus 2.B2): rich is reported 0.21 s after the step (dead time), lean again after 1.25 s, afterwards mean $q = 0.9000$. At +10 % the mixture becomes lean, q ramps up and settles around mean $q = 1.098$. The amplitude of the limit cycle in λ is unchanged. Integral action removes any constant disturbance – this is the purpose of the lambda loop (adaptation of fuel metering errors, air leaks, ageing).

d) $P = 0.03$, $r = 0.2$: period 0.70 s, λ 0.958…1.045; $P = 0.025$, $r = 0.1$: period 0.58 s, λ 0.978…1.027 (model values). The jump immediately undoes most of the overshoot that accumulated during the dead time; therefore a smaller ramp suffices and the amplitude halves while the period even decreases. The oscillation cannot disappear: the sensor delivers only the *sign* of $\lambda - 1$ (relay characteristic), so the controller must keep moving to find out where $\lambda = 1$ is – relay + integrator + dead time always yield a limit cycle of period $\approx 4T_l$. A PI controller with a fixed operating point would need a *proportional* measure of $\lambda - 1$: a wide-band (linear) lambda sensor. With the switching sensor, the catalytic converter's oxygen storage tolerates the small, fast oscillation – one reason why the period and amplitude must be kept small, i.e. why T_d matters.

Deliverables

2.1: property table, proofs. 2.2: sketch, plot of λ and U over 5 s, period over T_d (table + plot), results of c) and d). Bonus 2.B1 (optional): test program, property table with numerical evidence, $h[n]$ of the LTI systems. Bonus 2.B2 (optional): program, values of τ, response times, prediction formula and comparison table.

Bonus: JSLinux (optional)

For interested students: the following exercises identify unknown systems by experiment and simulate the lambda path offline in C. Bonus Exercise 2.B2 c) derives the limit cycle of Exercise 2.2 in detail.

Setup

The following exercises run in JSLinux, a Linux system in the browser. If you have not done Bonus Exercise 0.B1, set it up now (about 10 min):

1. Open `https://bellard.org/jslinux`, start the *x86 Alpine Linux* (console) virtual machine and check the compiler with `gcc -version` (or `tcc -v`).
2. Upload the course headers `engine_signals.h`, `asciiplot.h`, `csvio.h` (`code/common/`) and the files of the exercise (e.g. `code/jslinux/sheet02/`) with the upload button below the terminal; check with `ls` that they arrived. Headers and program must be in the same directory.
3. Compile and run, e.g.

```
gcc -O2 -o blackbox blackbox.c -lm
./blackbox
```

4. The VM forgets all files when the page is reloaded: keep your sources on your own computer and upload them again. Text from Wokwi (Serial Monitor) gets into the VM with `cat > data.csv`, paste, `Ctrl-D`; `export_file data.csv` downloads a file from the VM to your computer.

Exercise 2.B1 Black-box system identification

JSLINUX

The header `code/jslinux/sheet02/mystery.h` contains five unknown systems

$$\texttt{void Sk(const double *x, double *y, int N);} \quad k = 1, \ldots, 5$$

(input $x[0..N-1]$, $x[n] = 0$ outside, output $y[0..N-1]$). One of them is LTI; the others violate exactly one or two of the properties of Exercise 2.1. **Do not read the header** – find the properties by experiments, as you would with a supplier's ECU function delivered as object code. The program `code/jslinux/sheet02/blackbox.c` plots the responses to unit impulses at $n_0 = 30$ and $n_0 = 31$ and contains a (weak) homogeneity test with scaled impulses as an example.

a) Superposition test: compare $S\{ax_1+bx_2\}$ with $aS\{x_1\}+bS\{x_2\}$ for random signals and random weights, at amplitude 1 and 0.01. Why is the impulse test of the template not sufficient, and why two amplitudes?

b) Shift test: compare $S\{x[n-k]\}$ with $y[n-k]$ for $k = 1 \ldots 4$. Why are the first and last samples of the test signal set to zero?

c) Causality test: two inputs that agree for $n < n_0$ and differ afterwards.

d) Memory test: two inputs that differ only at n_0. Why must the test be repeated for several n_0?

e) Report a property table with the numerical evidence (largest deviation of each test). For the LTI systems determine $h[n]$ from the plots and give the difference equation. Why does an "impulse response" not characterise the other systems?

Solution

Output of `code/solutions/jslinux/sheet02/blackbox.c` (deviations of the linearity tests relative to the output amplitude):

```
sys  linear(A=1) linear(A=0.01) imp-scale   shift      causal     memory
S1    0.00e+00   0.00e+00       0.00e+00   1.96e+00   0.00e+00   1.00e+00
S2    3.78e-16   3.49e-16       0.00e+00   0.00e+00   0.00e+00   2.50e-01
S3    1.55e+00   3.58e-04       1.29e+00   0.00e+00   0.00e+00   0.00e+00
S4    0.00e+00   2.47e-16       0.00e+00   0.00e+00   4.61e-01   2.50e-01
S5    8.43e-01   8.48e-03       0.00e+00   0.00e+00   0.00e+00   6.21e-01

sys  linear  time-inv  causal  memoryless
S1   yes     NO       yes     NO
S2   yes     yes      yes     NO
S3   NO      yes      yes     yes
S4   yes     yes      NO      NO
S5   NO      yes      yes     NO
```

Deviations of 10^{-16} are rounding errors (the threshold is 10^{-9}). The systems are (from `code/solutions/jslinux/sheet02/mystery_gen.c`):

	system	properties
S1	$y[n] = x[n - (n \bmod 3)]$ (sample & hold every 3 samples)	linear, time-variant, causal, memory
S2	$y[n] = 0.75\,y[n-1] + 0.25\,x[n-1]$	LTI, causal, memory (IIR)
S3	$y[n] = \tanh(x[n])$	nonlinear, memoryless, TI, causal
S4	$y[n] = 0.25x[n-1] + 0.5x[n] + 0.25x[n+1]$	LTI, *non-causal*, memory
S5	$y[n] = 0.625x[n] + 0.375x[n-1] + 0.25\,x[n]x[n-1]$	nonlinear, TI, causal, memory

a) The impulse homogeneity test detects only S3. S5 passes it, because the product term $x[n]x[n-1]$ vanishes for any *single* impulse: an impulse never excites interactions between samples. Random signals with random weights excite all such terms. S3 (tanh) is almost linear for small amplitudes ($\tanh x \approx x - x^3/3$): relative deviation $3.6 \cdot 10^{-4}$ at $A = 0.01$ versus 1.55 at $A = 1$ – small-signal linearisation; a test at small amplitude alone could miss the nonlinearity if the tolerance is too coarse. Similarly S5's deviation drops from 0.84 to $8.5 \cdot 10^{-3}$ (quadratic term $\propto A$ relative to the linear part).
b) The shifted copy must not lose samples at the end or receive the implicit zeros of the boundary at the start; otherwise a non-causal LTI system such as S4 (which reads $x[n+1]$) shows boundary deviations and is wrongly classified as time-variant. S1 shows deviations up to 1.96: shifting the input by one sample changes which samples are held.
c) Only S4 reacts before the inputs differ (deviation 0.46 at $n = n_0 - 1$, weight 0.25 of $x[n+1]$).
d) For S1 a change at n_0 with $n_0 \bmod 3 \neq 0$ has *no* effect at all (the sample is never read), with $n_0 \bmod 3 = 0$ it affects $n_0 \ldots n_0 + 2$. A single n_0 may therefore report "memoryless" or even "no effect". In general: a test can only falsify a property; a passed test is evidence, not proof – use many random trials.
e) LTI systems: S2: $h[n] = 0$ for $n \leq 0$, $h[n] = 0.25 \cdot 0.75^{\,n-1}$ for $n \geq 1$ (plot: 0.25, 0.1875, 0.141, … starting one sample after the impulse) $\Rightarrow y[n] = 0.75y[n-1] + 0.25x[n-1]$, an exponential smoother with one sample delay. S4: $h[-1] = 0.25$, $h[0] = 0.5$, $h[1] = 0.25$ (the plot for $n_0 = 30$ shows values at 29, 30, 31) $\Rightarrow$ centred, non-causal smoother. For the other systems the response to an impulse depends on *where* the impulse is (S1: three ones for $n_0 = 30$, nothing for $n_0 = 31$) or on *how large* it is and which other samples are present (S3, S5): $y = h * x$ (Chapter 3) requires linearity *and* time invariance.

```c
/* ---- (a) superposition: S{a x1 + b x2} = a S{x1} + b S{x2} ? ---------
   returns the largest deviation relative to the output amplitude       */
static double test_linear(system_t S, double amp)
{
    double x1[N], x2[N], x3[N], y1[N], y2[N], y3[N], d[N], e = 0.0, r;
    int t, n;
    for (t = 0; t < NTRY; t++) {
        double a = 2.0 * urand(), b = 2.0 * urand();
        rand_signal(x1, amp, 0);
        rand_signal(x2, amp, 0);
```

```
        for (n = 0; n < N; n++) x3[n] = a * x1[n] + b * x2[n];
        S(x1, y1, N); S(x2, y2, N); S(x3, y3, N);
        for (n = 0; n < N; n++) d[n] = y3[n] - (a * y1[n] + b * y2[n]);
        r = maxabs(d, N) / (maxabs(y3, N) + 1e-300);
        if (r > e) e = r;
    }
    return e;
}
```

Exercise 2.B2 The engine as a dynamic system JSLINUX

Program: `code/jslinux/sheet02/engine_dyn.c`.

a) *Crankshaft inertia.* With fuel quantity q (1 = nominal), the engine speed obeys $\tau\,\dot{n} = Kq - n$, $\tau = 0.4\,\mathrm{s}$, $K = 2000\,\mathrm{rpm}$. Simulate a fuel step $1 \to 1.5$ at $t = 0.5\,\mathrm{s}$ with the Euler method ($T = 1\,\mathrm{ms}$) and with the exact discretisation $n[k] = a\,n[k-1] + (1-a)Kq[k-1]$, $a = \mathrm{e}^{-T/\tau}$ (derive it!). Determine τ from the 63 % point and compare both methods with the analytic solution at $t = 0.6\,\mathrm{s}$. Is the engine (fuel → speed) static or dynamic?

b) *Lambda path.* Implement `lp_step()` (one step = 1 ms) – it is the model inside the Wokwi chip `lambda-probe`: $q \to$ wall film (first-order lag, 20 ms) $\to \lambda = (1 + a_{\mathrm{air}})/q_f \to$ transport delay T_d → sensor lag (30 ms) $\to U = 0.5 + 0.4\tanh((1-\lambda)/0.005)$. Classify each block (static/dynamic, linear, TI). Simulate fuel steps $1.00 \to 1.10$ and $1.00 \to 1.20$ ($a_{\mathrm{air}} = +5\,\%$, $T_d = 200\,\mathrm{ms}$) and measure the time until U crosses 0.45 V. Which observations prove that the chain is nonlinear? Under which operating condition is it not time-invariant on a real engine?

c) *Closed loop – offline model of Exercise 2.2.* A two-point controller with integrator changes q with slope $-r$ while the sensor reports rich ($U > 0.45\,\mathrm{V}$) and $+r$ while lean (period 10 ms, 8-bit PWM). Derive the period and the λ amplitude of the resulting oscillation as functions of T_d, r and the lag time constants (hint: a ramp passing a first-order lag τ is delayed by τ). Implement the controller in `closed_loop()`, compare with your prediction for $T_d = 50\ldots800\,\mathrm{ms}$, and explain what happens to the mean of q when the unmetered air changes. Extend the prediction to a controller with an additional jump $\pm P$ at every switching of the sensor.

Solution

a) Exact discretisation: with q constant over $[kT - T, kT)$ the ODE solution is $n(kT) = \mathrm{e}^{-T/\tau}n(kT-T) + (1 - \mathrm{e}^{-T/\tau})Kq$. Output:

```
(a) fuel step 1.0 -> 1.5 at t = 0.5 s: n = 2000 -> 2998.1 rpm (t = 3 s)
    63% point reached at t = 0.900 s -> tau = 0.400 s
    n(0.6 s): Euler 2221.44, exact 2221.20, analytic 2221.20 rpm
    difference equation: n[k] = 0.997503 n[k-1] + 0.002497 * K q[k-1]
```

The exact discretisation reproduces the analytic solution $2000 + 1000(1 - \mathrm{e}^{-0.1/0.4}) = 2221.20\,\mathrm{rpm}$; Euler ($a = 1 - T/\tau = 0.9975$) is off by 0.24 rpm – negligible at $T/\tau = 1/400$. The speed approaches 3000 rpm exponentially; it depends on the past fuel input ⇒ dynamic system (with memory), LTI in this model. The difference equation is the exponential smoother of Sheet 3.
b) Blocks: wall film – dynamic, linear, TI; $\lambda = (1+a)/q_f$ – static, *nonlinear*; transport delay – dynamic (memory), linear, TI; sensor lag – dynamic, linear, TI; switching characteristic – static, strongly nonlinear (slope 80 V per unit λ at $\lambda = 1$, saturating at 0.1/0.9 V).

```
(b) fuel step 1.00 -> 1.10 at t = 100 ms (air +5 %, Td = 200 ms)
    lambda 1.0500 -> 0.9545, crosses 1 after 13 ms (wall film)
    sensor crosses 0.45 V after 240 ms
    with step 1.00 -> 1.20 (double size): crosses after 223 ms, final U = 0.900 V (step 1.10:
    0.900 V)
```

Response time = wall film (λ crosses 1 after 13 ms, theory $20\,\mathrm{ms}\cdot\ln 2 = 13.9\,\mathrm{ms}$) + 200 ms delay + sensor lag ($\approx 27\,\mathrm{ms}$). Nonlinearity: doubling the input step does not double the output (final

value 0.9 V in both cases), and the response *time* depends on the step size (240 vs. 223 ms) – impossible for an LTI system, whose step response only scales. The sensor tells only "rich" or "lean", not *how* rich. On a real engine T_d is the gas travel time to the sensor, roughly $\propto 1/(\text{mass flow})$: at changing speed/load the system is time-variant.

c) Let $T_l = T_d + \tau_f + \tau_s$ be the effective dead time of the loop and $q^* = 1 + a$ the stoichiometric fuel quantity. When the sensor switches to rich, q has already passed q^* by rT_l. The controller now ramps down; q reaches q^* after T_l, and the sensor sees it another T_l later. Hence

$$T_{\text{period}} = 4\,T_l, \qquad q = q^* \pm rT_l, \qquad \lambda \approx 1 \mp \frac{rT_l}{q^*}.$$

With an additional jump $P < rT_l$ the time to reach q^* is shortened by P/r: $T_{\text{period}} = 4T_l - 2P/r$ (same peak deviation). Simulation ($r = 0.2/\text{s}$, +5 % air, $T_l = T_d + 50\,\text{ms}$):

T_d / ms	period (sim.)	$4T_l$	λ range (sim.)
50	0.38 s	0.40 s	0.985 … 1.017
100	0.58 s	0.60 s	0.977 … 1.028
200	0.98 s	1.00 s	0.958 … 1.046
400	1.78 s	1.80 s	0.924 … 1.091
800	3.38 s	3.40 s	0.864 … 1.193

The prediction fits within 2–5 % (the effective lag is slightly smaller than 50 ms because the sensor switches before λ_s exactly reaches 1). Unmetered air 0/ + 5/ + 10 %: mean q = 0.9989 / 1.0495 / 1.0986 $\approx 1 + a$ – the integrator drives the *mean* mixture to $\lambda = 1$ whatever the disturbance (integral action), period unchanged (0.98 s). With $P = 0.03$: period 0.70 s (prediction $1.0 - 0.3 = 0.70$ s), λ 0.958 … 1.045; with $r = 0.1/\text{s}$, $P = 0.025$: period 0.58 s, λ 0.978 … 1.027 (prediction 0.5 s, ±2.4 %): the jump allows a smaller ramp, i.e. a smaller amplitude at the same or a higher frequency.

```c
static double lp_step(lam_path_t *p, double q, double *lam_true)
{
    double lam, lam_d;
    p->qf += (q - p->qf) / 20.0;                        /* wall film, 20 ms  */
    lam = (1.0 + p->air) / (p->qf > 0.05 ? p->qf : 0.05);   /* static, NL */
    if (lam > 2.0) lam = 2.0;
    if (lam < 0.5) lam = 0.5;
    p->buf[p->w] = lam;                                  /* transport delay */
    lam_d = p->buf[(p->w - p->delay + MAXD) % MAXD];
    p->w = (p->w + 1) % MAXD;
    p->ls += (lam_d - p->ls) / 30.0;                     /* sensor lag, 30 ms */
    *lam_true = lam;
    return es_lambda_voltage(p->ls);                     /* static, NL     */
}
```